

Deploying QuizzDjangoForms to Google Cloud Run

Step-by-step record of the deployment performed for the QuizzDjangoForms Django project — Cloud Run + Cloud SQL (Postgres)

Contents

1. Prerequisites
2. Google Cloud account & billing setup
3. Create the GCP project
4. Enable required APIs
5. Create the Cloud SQL (Postgres) database
6. Update the Django project for production
7. Add production dependencies
8. Create the Dockerfile
9. Store secrets in Secret Manager
10. Deploy to Cloud Run
11. Create the Django admin superuser
12. Verify the deployment
13. Redeploying after future changes

1. Prerequisites

Tools used on the local machine:

- **Google Cloud SDK (`gcloud`)** — already installed
- **uv** — Python package/dependency manager used by this project
- A Google account to own the GCP project (`alex.finder.89@gmail.com`)

Confirm `gcloud` is authenticated with the correct account:

```
gcloud auth list
gcloud config list
```

If the wrong account/project is active, log in with the correct account:

```
gcloud auth login
```

then select it:

```
gcloud config set account alex.finder.89@gmail.com
```

2. Google Cloud account & billing setup

Cloud Run and Cloud SQL both require a project linked to an **active (open)** billing account.

```
gcloud billing accounts list
```

If your only billing account shows `open: false`, it has been closed/disabled. Reactivating a billing account or creating a new one requires the Cloud Console UI (a payment method must be attached):

<https://console.cloud.google.com/billing>

Once an account shows `open: true`, note its `ACCOUNT_ID` (format `XXXXXX-XXXXXX-XXXXXX`) — it is needed in the next step.

3. Create the GCP project

3.1 Create the project

```
gcloud projects create quizz-django-forms --name="QuizzDjangoForms"
```

3.2 Set it as the active project and default region

```
gcloud config set project quizz-django-forms  
gcloud config set run/region europe-west1
```

3.3 Link the billing account

```
gcloud billing projects link quizz-django-forms \  
--billing-account=<YOUR_BILLING_ACCOUNT_ID>
```

4. Enable required APIs

Cloud Run, Cloud SQL, Artifact Registry (image storage), Cloud Build (image builds), Secret Manager (secrets) and Compute (networking) APIs are needed:

```
gcloud services enable run.googleapis.com sqladmin.googleapis.com \  
artifactregistry.googleapis.com cloudbuild.googleapis.com \  
secretmanager.googleapis.com compute.googleapis.com
```

5. Create the Cloud SQL (Postgres) database

5.1 Create the instance

The smallest Postgres tier (`db-f1-micro`) requires the `ENTERPRISE` edition flag:

```
gcloud sql instances create quizz-db \  
  --database-version=POSTGRES_16 \  
  --edition=ENTERPRISE \  
  --tier=db-f1-micro \  
  --region=europe-west1 \  
  --storage-size=10 \  
  --storage-type=HDD \  
  --no-backup
```

This step takes roughly 5–10 minutes to provision.

5.2 Create the database and application user

```
gcloud sql databases create quizzdb --instance=quizz-db  
  
gcloud sql users create quizzuser --instance=quizz-db \  
  --password="<GENERATED_DB_PASSWORD>"
```

5.3 Note the connection name

Used later to connect Cloud Run to this instance via a Unix socket:

```
gcloud sql instances describe quizz-db --format="value(connectionName)"  
# → quizz-django-forms:europe-west1:quizz-db
```

6. Update the Django project for production

Edits made to `quizz_web_project/settings.py`:

6.1 Read secrets/config from environment variables

```
import os
from pathlib import Path

BASE_DIR = Path(__file__).resolve().parent.parent

SECRET_KEY = os.environ.get(
    'SECRET_KEY',
    'django-insecure-...', # local-dev fallback only
)

DEBUG = os.environ.get('DEBUG', 'True') == 'True'

ALLOWED_HOSTS = os.environ.get(
    'ALLOWED_HOSTS', '192.168.100.130,localhost,127.0.0.1'
).split(',')

CSRF_TRUSTED_ORIGINS = [
    o for o in os.environ.get('CSRF_TRUSTED_ORIGINS', '').split(',') if o
]

# Cloud Run terminates TLS and forwards over HTTP with this header.
SECURE_PROXY_SSL_HEADER = ('HTTP_X_FORWARDED_PROTO', 'https')
```

6.2 Add WhiteNoise for static files

```
MIDDLEWARE = [
    'django.middleware.security.SecurityMiddleware',
    'whitenoise.middleware.WhiteNoiseMiddleware',
    'django.contrib.sessions.middleware.SessionMiddleware',
    ...
]
```

6.3 Switch to Postgres when DB_NAME is set (Cloud SQL), else SQLite

```
if os.environ.get('DB_NAME'):
    DATABASES = {
        'default': {
            'ENGINE': 'django.db.backends.postgresql',
            'NAME': os.environ['DB_NAME'],
            'USER': os.environ['DB_USER'],
            'PASSWORD': os.environ['DB_PASSWORD'],
            'HOST': os.environ['DB_HOST'],
            'PORT': os.environ.get('DB_PORT', '5432'),
        }
    }
else:
    DATABASES = {
        'default': {
            'ENGINE': 'django.db.backends.sqlite3',
            'NAME': BASE_DIR / 'db.sqlite3',
        }
    }
```

6.4 Configure static file storage

```
STATIC_URL = 'static/'
STATIC_ROOT = BASE_DIR / 'staticfiles'

STORAGES = {
    'staticfiles': {
        'BACKEND': 'whitenoise.storage.CompressedManifestStaticFilesStorage',
    },
}
```

Local development is unaffected: with no `DB_NAME` env var set and `DEBUG` defaulting to `True`, `runserver.bat` keeps using SQLite.

7. Add production dependencies

Added with `uv` (updates `pyproject.toml` and `uv.lock`):

```
uv add gunicorn whitenoise "psycogp[binary]"
```

Package	Purpose
<code>gunicorn</code>	Production WSGI server
<code>whitenoise</code>	Serve static files directly from the app container
<code>psycogp[binary]</code>	PostgreSQL database driver

8. Create the Dockerfile

8.1 Dockerfile

```
FROM python:3.14-slim

ENV PYTHONDONTWRITEBYTECODE=1 \
    PYTHONUNBUFFERED=1

WORKDIR /app

COPY --from=ghcr.io/astral-sh/uv:latest /uv /uvx /bin/

COPY pyproject.toml uv.lock ./
RUN uv sync --frozen --no-dev --no-install-project

COPY . .
RUN uv sync --frozen --no-dev

ENV PATH="/app/.venv/bin:$PATH"

RUN python manage.py collectstatic --noinput

EXPOSE 8080

CMD ["sh", "-c", "python manage.py migrate --noinput && gunicorn\
quizz_web_project.wsgi:application --bind 0.0.0.0:8080 --workers 2"]
```

8.2 .dockerignore

```
.venv
__pycache__
*.pyc
db.sqlite3
db.sqlite3-journal
.git
.gitignore
staticfiles
*.bat
README.md
LICENSE
```

Running `migrate` on every container start is simple and idempotent — suitable for a small app. For larger projects, run migrations as a separate Cloud Run Job/step instead.

9. Store secrets in Secret Manager

Generate a Django secret key and store it, plus the database password:

```
SECRET_KEY=$(uv run python -c \  
    "from django.core.management.utils import get_random_secret_key;  
    print(get_random_secret_key())")  
echo -n "$SECRET_KEY" | gcloud secrets create django-secret-key --data-file=-  
  
echo -n "<DB_PASSWORD>" | gcloud secrets create db-password --data-file=-
```


10. Deploy to Cloud Run

10.1 Build and deploy from source

```
gcloud run deploy quizz-django-forms \
  --source . \
  --region europe-west1 \
  --allow-unauthenticated \
  --add-cloudsql-instances quizz-django-forms:europe-west1:quizz-db \
  --set-env-vars "DEBUG=False,ALLOWED_HOSTS=.run.app,\
  CSRF_TRUSTED_ORIGINS=https://*.run.app,DB_NAME=quizzdb,DB_USER=quizzuser,\
  DB_HOST=/cloudsql/quizz-django-forms:europe-west1:quizz-db" \
  --set-secrets "SECRET_KEY=django-secret-key:latest,DB_PASSWORD=db-password:latest"
```

This builds the Dockerfile via Cloud Build, pushes the image to Artifact Registry (`cloud-run-source-deploy` repo, created automatically), and deploys it.

10.2 Grant the Cloud Run service account access to the secrets

The first deploy failed with: Permission denied on secret ... for Revision service account
<PROJECT_NUMBER>-compute@developer.gserviceaccount.com

Fix by granting the **Secret Manager Secret Accessor** role to the Compute default service account that Cloud Run uses:

```
gcloud projects add-iam-policy-binding quizz-django-forms \
  --member="serviceAccount:<PROJECT_NUMBER>-compute@developer.gserviceaccount.com" \
  --role="roles/secretmanager.secretAccessor"
```

10.3 Re-deploy using the already-built image

No need to rebuild — reuse the image from the previous (failed-deploy) build:

```
gcloud builds list --region=europe-west1 --limit=1 --format="value(images)"
# → europe-west1-docker.pkg.dev/quizz-django-forms/cloud-run-source-deploy/quizz-django-
forms:latest

gcloud run deploy quizz-django-forms \
  --image europe-west1-docker.pkg.dev/quizz-django-forms/cloud-run-source-deploy/quizz-django-
forms:latest \
  --region europe-west1 \
  --allow-unauthenticated \
  --add-cloudsql-instances quizz-django-forms:europe-west1:quizz-db \
  --set-env-vars "DEBUG=False,ALLOWED_HOSTS=.run.app,\
  CSRF_TRUSTED_ORIGINS=https://*.run.app,DB_NAME=quizzdb,DB_USER=quizzuser,\
  DB_HOST=/cloudsql/quizz-django-forms:europe-west1:quizz-db" \
  --set-secrets "SECRET_KEY=django-secret-key:latest,DB_PASSWORD=db-password:latest"
```

Result:

```
Service [quizz-django-forms] revision [quizz-django-forms-00002-ttn] has been deployed
and is serving 100 percent of traffic.
Service URL: https://quizz-django-forms-530048627940.europe-west1.run.app
```

11. Create the Django admin superuser

Since the container has no interactive shell access, a one-off **Cloud Run Job** is used to run `createsuperuser --noinput`.

11.1 Store the admin password as a secret

```
echo -n "<ADMIN_PASSWORD>" | gcloud secrets create django-admin-password --data-file=-
```

11.2 Create the job

```
gcloud run jobs create create-superuser \
  --image europe-west1-docker.pkg.dev/quizz-django-forms/cloud-run-source-deploy/quizz-django-
forms:latest \
  --region europe-west1 \
  --set-cloudsql-instances quizz-django-forms:europe-west1:quizz-db \
  --set-env-vars "DEBUG=False,DB_NAME=quizzdb,DB_USER=quizzuser,\
DB_HOST=/cloudsql/quizz-django-forms:europe-west1:quizz-db,\
DJANGO_SUPERUSER_USERNAME=admin,DJANGO_SUPERUSER_EMAIL=alex.finder.89@gmail.com" \
  --set-secrets "SECRET_KEY=django-secret-key:latest,DB_PASSWORD=db-password:latest,\
DJANGO_SUPERUSER_PASSWORD=django-admin-password:latest" \
  --command "python" \
  --args "manage.py,createsuperuser,--noinput"
```

11.3 Run it

```
gcloud run jobs execute create-superuser --region europe-west1 --wait
```

11.4 Clean up the one-off job

```
gcloud run jobs delete create-superuser --region europe-west1 --quiet
```

After logging in as `admin` and changing the password, delete the temporary secret:

```
gcloud secrets delete django-admin-password
```

12. Verify the deployment

```
curl -s -o /dev/null -w "%{http_code}\n" https://quizz-django-forms-530048627940.europe-west1.run.app/  
# → 302 (redirect to login – expected)  
  
curl -s -o /dev/null -w "%{http_code}\n" https://quizz-django-forms-530048627940.europe-west1.run.app/accounts/login/  
# → 200  
  
curl -s -o /dev/null -w "%{http_code}\n" https://quizz-django-forms-530048627940.europe-west1.run.app/admin/login/  
# → 200
```

You can also check the service status at any time:

```
gcloud run services describe quizz-django-forms --region europe-west1 \  
--format="value(status.url,status.conditions[0].status,status.conditions[0].type)"
```

13. Redeploying after future changes

From the project root, after committing your code changes:

```
gcloud run deploy quizz-django-forms --source . --region europe-west1
```

This rebuilds the Docker image via Cloud Build, pushes it to Artifact Registry, and rolls out a new revision. Migrations run automatically on container start.

Resources created in project `quizz-django-forms`

Resource	Name	Region
Cloud Run service	quizz-django-forms	europe-west1
Cloud SQL instance (Postgres 16, db-f1-micro)	quizz-db	europe-west1
Cloud SQL database	quizzdb	—
Cloud SQL user	quizzuser	—
Artifact Registry repo	cloud-run-source-deploy	europe-west1
Secret Manager secrets	django-secret-key, db-password	—
Billing account	Alex Finder Billing Personal 1	—

Cost note: Cloud Run scales to zero when idle (minimal cost), but the Cloud SQL `db-f1-micro` instance runs continuously and incurs an hourly charge. Stop or delete it when not needed:

```
gcloud sql instances patch quizz-db --activation-policy=NEVER # stop
gcloud sql instances delete quizz-db                        # delete permanently
```